

# Experiment 5

## Decision Tree and Random Forest: A Comparative Classification Study

Jayasri Murugesan

August 2026

### 1 Objective

- To implement a Decision Tree classifier.
- To extend the Decision Tree into a Random Forest ensemble model.
- To study the impact of hyperparameters on overfitting and generalization.
- To select optimal hyperparameters using 5-Fold Cross-Validation.
- To compare single-tree and ensemble-tree models.

### 2 Dataset Description

The Wisconsin Diagnostic Breast Cancer dataset was used in this experiment. The dataset contains 569 samples and 30 numerical features related to the characteristics of breast cell nuclei. The target variable represents the diagnosis of a tumor as either benign or malignant.

### 3 Preprocessing Steps

- Loaded the dataset.
- Assigned the feature and target column names.
- Removed the ID column.
- Encoded the diagnosis values as 0 for benign and 1 for malignant.
- Checked the number of samples and features.
- Performed exploratory data analysis.
- Split the dataset into training and testing sets using stratification.

### 4 Implementation Details

The following machine learning models were implemented:

- Decision Tree Classifier
- Random Forest Classifier

GridSearchCV with five-fold Stratified K-Fold cross-validation was used for hyperparameter tuning. The models were evaluated using the following performance metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC Curve
- Area Under the Curve (AUC)

## 5 Code

The complete Python implementation and execution results are attached below.

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, co
```

```
In [3]: ff=pd.read_csv("C:/Users/DHARSHINI/Downloads/sem5/machine-learning/experiment5/wdbc
ff.head()
```

```
Out[3]:
```

|   | 0        | 1 | 2     | 3     | 4      | 5      | 6       | 7       | 8      | 9       | ... | 22    |
|---|----------|---|-------|-------|--------|--------|---------|---------|--------|---------|-----|-------|
| 0 | 842302   | M | 17.99 | 10.38 | 122.80 | 1001.0 | 0.11840 | 0.27760 | 0.3001 | 0.14710 | ... | 25.38 |
| 1 | 842517   | M | 20.57 | 17.77 | 132.90 | 1326.0 | 0.08474 | 0.07864 | 0.0869 | 0.07017 | ... | 24.99 |
| 2 | 84300903 | M | 19.69 | 21.25 | 130.00 | 1203.0 | 0.10960 | 0.15990 | 0.1974 | 0.12790 | ... | 23.57 |
| 3 | 84348301 | M | 11.42 | 20.38 | 77.58  | 386.1  | 0.14250 | 0.28390 | 0.2414 | 0.10520 | ... | 14.91 |
| 4 | 84358402 | M | 20.29 | 14.34 | 135.10 | 1297.0 | 0.10030 | 0.13280 | 0.1980 | 0.10430 | ... | 22.54 |

5 rows × 32 columns



```
In [4]: X=ff.drop(1,axis=1)
y=ff[1]

print("Dataset Shape :", X.shape)
print("Target Shape :", y.shape)
```

Dataset Shape : (569, 31)

Target Shape : (569,)

```
In [5]: encoder = LabelEncoder()
y = encoder.fit_transform(y)

print("\nClass Distribution")
print(pd.Series(y).value_counts())
```

Class Distribution

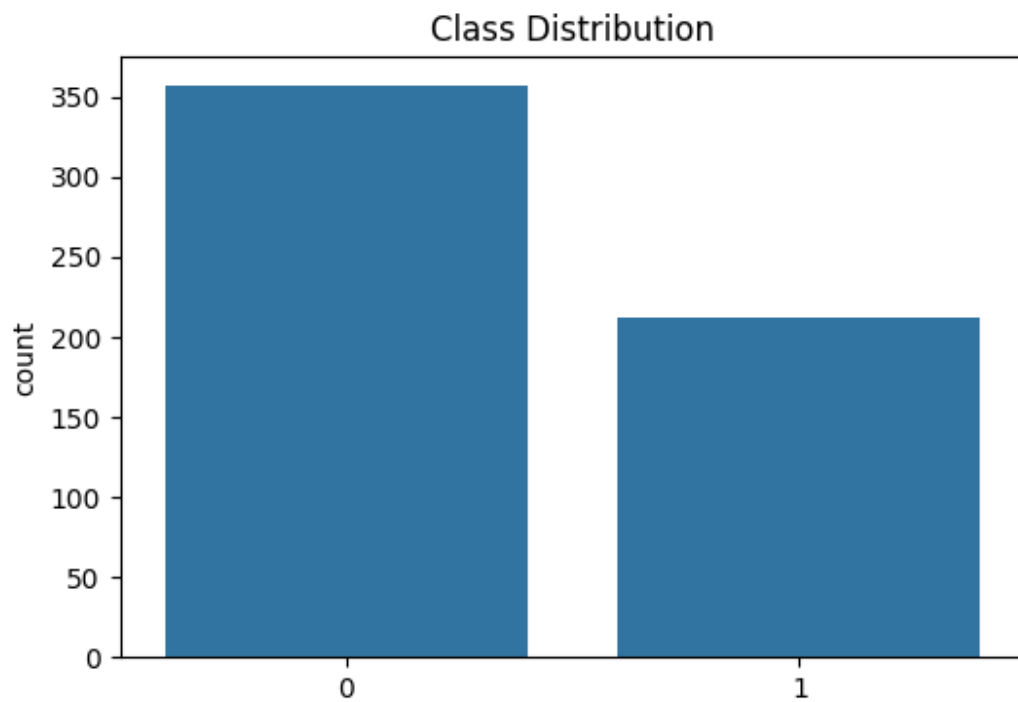
0 357

1 212

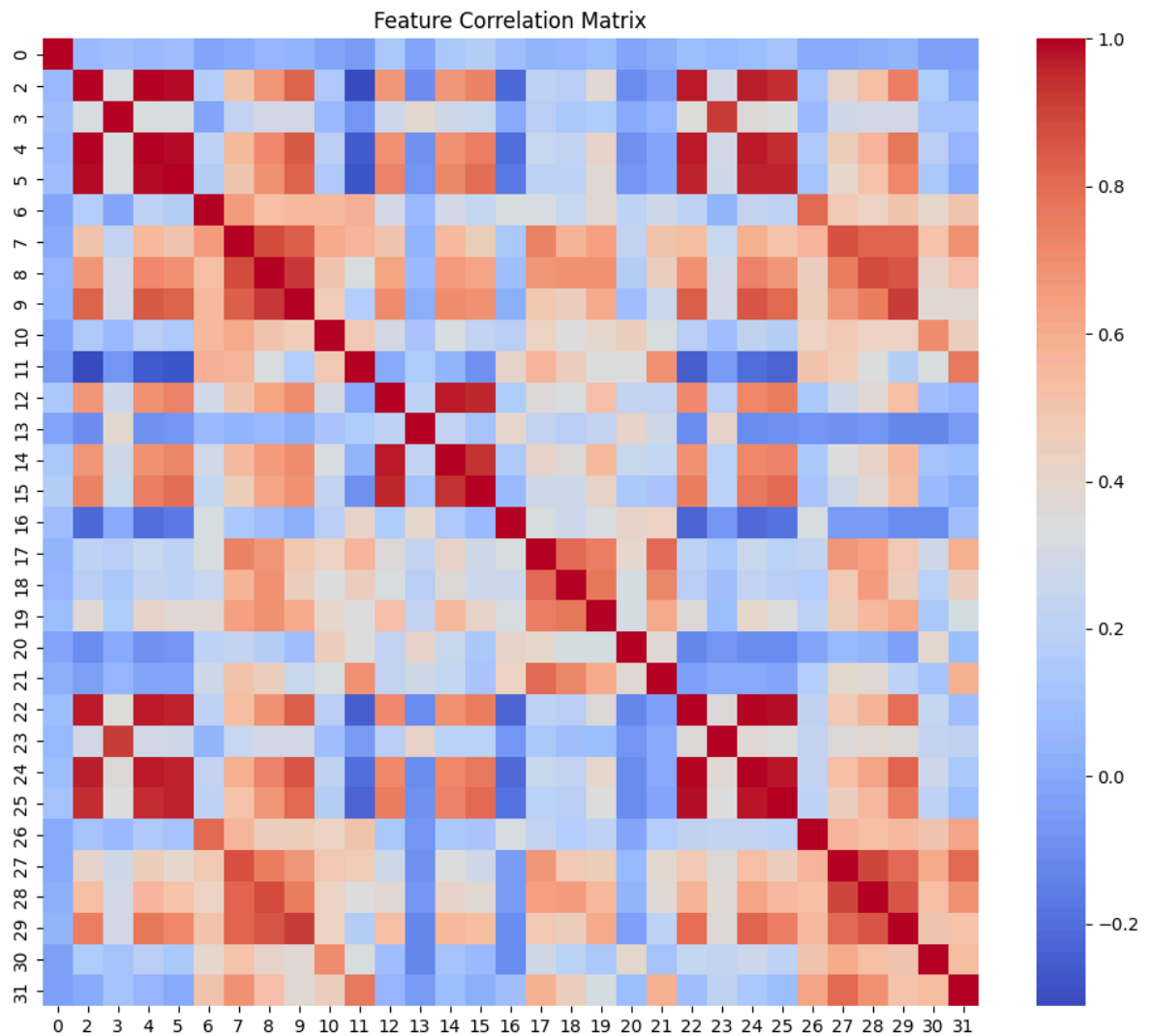
Name: count, dtype: int64

```
In [6]: plt.figure(figsize=(6,4))
sns.countplot(x=y)
```

```
plt.title("Class Distribution")  
plt.show()
```



```
In [7]: plt.figure(figsize=(12,10))  
sns.heatmap(X.corr(), cmap='coolwarm')  
plt.title("Feature Correlation Matrix")  
plt.show()
```



```
In [8]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )
```

```
In [9]: dt = DecisionTreeClassifier(random_state=42)

dt_params = {
    'criterion':['gini','entropy'],
    'max_depth':[3,5,7,10,None],
    'min_samples_split':[2,5,10],
    'min_samples_leaf':[1,2,4]
}
```

```
In [10]: dt_grid = GridSearchCV(
        dt,
        dt_params,
        cv=5,
```

```

        scoring='accuracy',
        n_jobs=-1
    )

    dt_grid.fit(X_train,y_train)

    best_dt = dt_grid.best_estimator_

    print("\nBest Decision Tree Parameters")
    print(dt_grid.best_params_)

```

Best Decision Tree Parameters

```
{'criterion': 'entropy', 'max_depth': 7, 'min_samples_leaf': 2, 'min_samples_split': 5}
```

In [11]: `dt_results = pd.DataFrame(dt_grid.cv_results_)`

```

table1 = dt_results[
    [
        'param_criterion',
        'param_max_depth',
        'mean_test_score'
    ]
]

table1.columns = [
    'Criterion',
    'Max Depth',
    'Avg CV Accuracy'
]

print("\nTABLE 1")
print(table1.head(15))

```

TABLE 1

|    | Criterion | Max Depth | Avg CV Accuracy |
|----|-----------|-----------|-----------------|
| 0  | gini      | 3         | 0.927473        |
| 1  | gini      | 3         | 0.927473        |
| 2  | gini      | 3         | 0.923077        |
| 3  | gini      | 3         | 0.920879        |
| 4  | gini      | 3         | 0.920879        |
| 5  | gini      | 3         | 0.923077        |
| 6  | gini      | 3         | 0.925275        |
| 7  | gini      | 3         | 0.925275        |
| 8  | gini      | 3         | 0.925275        |
| 9  | gini      | 5         | 0.927473        |
| 10 | gini      | 5         | 0.927473        |
| 11 | gini      | 5         | 0.920879        |
| 12 | gini      | 5         | 0.920879        |
| 13 | gini      | 5         | 0.925275        |
| 14 | gini      | 5         | 0.920879        |

In [12]: `rf = RandomForestClassifier(random_state=42)`

```

rf_params = {
    'n_estimators':[50,100,200],
    'max_depth':[5,10,15,None],

```

```

    'max_features':['sqrt','log2'],
    'bootstrap':[True,False]
}

```

```

In [13]: rf_grid = GridSearchCV(
            rf,
            rf_params,
            cv=5,
            scoring='accuracy',
            n_jobs=-1
        )

rf_grid.fit(X_train,y_train)

best_rf = rf_grid.best_estimator_

print("\nBest Random Forest Parameters")
print(rf_grid.best_params_)

```

Best Random Forest Parameters

```
{'bootstrap': False, 'max_depth': 10, 'max_features': 'sqrt', 'n_estimators': 200}
```

```

In [14]: rf_results = pd.DataFrame(rf_grid.cv_results_)

table2 = rf_results[
    [
        'param_n_estimators',
        'param_max_depth',
        'param_max_features',
        'mean_test_score'
    ]
]

table2.columns = [
    'n_estimators',
    'Max Depth',
    'Max Features',
    'Avg CV Accuracy'
]

print("\nTABLE 2")
print(table2.head(15))

```

TABLE 2

|    | n_estimators | Max Depth | Max Features | Avg CV Accuracy |
|----|--------------|-----------|--------------|-----------------|
| 0  | 50           | 5         | sqrt         | 0.949451        |
| 1  | 100          | 5         | sqrt         | 0.956044        |
| 2  | 200          | 5         | sqrt         | 0.953846        |
| 3  | 50           | 5         | log2         | 0.953846        |
| 4  | 100          | 5         | log2         | 0.956044        |
| 5  | 200          | 5         | log2         | 0.960440        |
| 6  | 50           | 10        | sqrt         | 0.953846        |
| 7  | 100          | 10        | sqrt         | 0.956044        |
| 8  | 200          | 10        | sqrt         | 0.956044        |
| 9  | 50           | 10        | log2         | 0.956044        |
| 10 | 100          | 10        | log2         | 0.958242        |
| 11 | 200          | 10        | log2         | 0.960440        |
| 12 | 50           | 15        | sqrt         | 0.953846        |
| 13 | 100          | 15        | sqrt         | 0.956044        |
| 14 | 200          | 15        | sqrt         | 0.956044        |

```
In [15]: dt_cv = cross_val_score(
    best_dt,
    X,
    y,
    cv=5,
    scoring='accuracy'
)

rf_cv = cross_val_score(
    best_rf,
    X,
    y,
    cv=5,
    scoring='accuracy'
)

comparison = pd.DataFrame({
    'Model': ['Decision Tree', 'Random Forest'],
    'Fold1': [dt_cv[0], rf_cv[0]],
    'Fold2': [dt_cv[1], rf_cv[1]],
    'Fold3': [dt_cv[2], rf_cv[2]],
    'Fold4': [dt_cv[3], rf_cv[3]],
    'Fold5': [dt_cv[4], rf_cv[4]],
    'Average': [dt_cv.mean(), rf_cv.mean()]
})

print("\nTABLE 3")
print(comparison)
```

TABLE 3

|   | Model         | Fold1    | Fold2    | Fold3    | Fold4    | Fold5    | Average  |
|---|---------------|----------|----------|----------|----------|----------|----------|
| 0 | Decision Tree | 0.903509 | 0.929825 | 0.947368 | 0.956140 | 0.946903 | 0.936749 |
| 1 | Random Forest | 0.929825 | 0.947368 | 0.982456 | 0.982456 | 0.964602 | 0.961341 |

```
In [16]: dt_pred = best_dt.predict(X_test)

print("\nDECISION TREE RESULTS")
```

```

print("Accuracy:",
      accuracy_score(y_test,dt_pred))

print("Precision:",
      precision_score(y_test,dt_pred))

print("Recall:",
      recall_score(y_test,dt_pred))

print("F1 Score:",
      f1_score(y_test,dt_pred))

print("\nClassification Report")
print(classification_report(y_test,dt_pred))

cm_dt = confusion_matrix(y_test,dt_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm_dt,
            annot=True,
            fmt='d')
plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

#### DECISION TREE RESULTS

Accuracy: 0.956140350877193

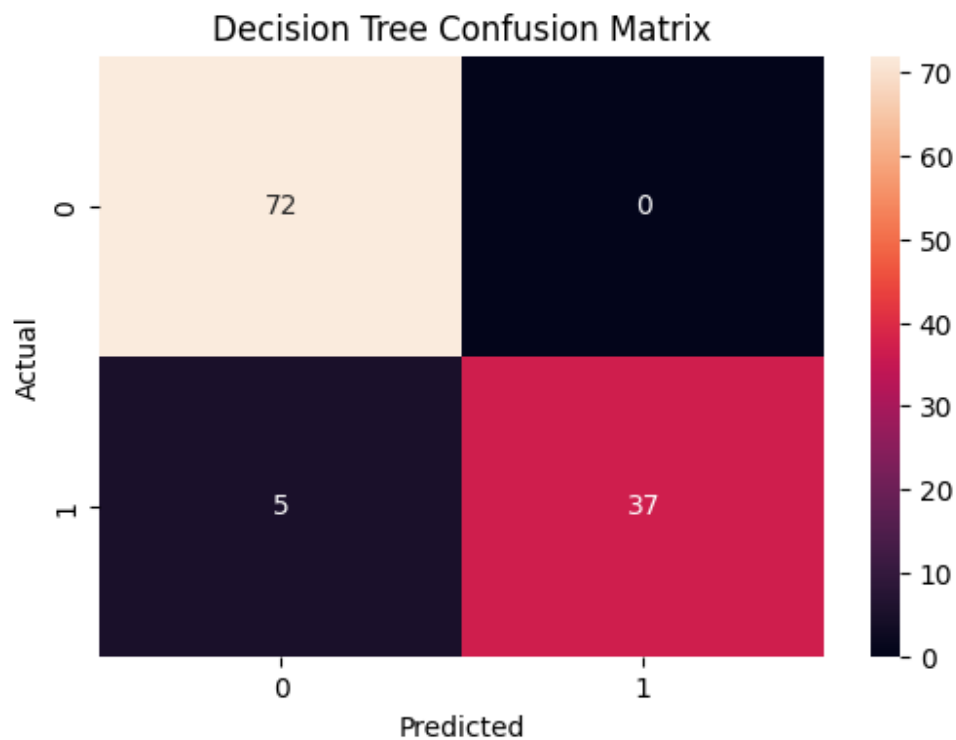
Precision: 1.0

Recall: 0.8809523809523809

F1 Score: 0.9367088607594937

#### Classification Report

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.94      | 1.00   | 0.97     | 72      |
| 1            | 1.00      | 0.88   | 0.94     | 42      |
| accuracy     |           |        | 0.96     | 114     |
| macro avg    | 0.97      | 0.94   | 0.95     | 114     |
| weighted avg | 0.96      | 0.96   | 0.96     | 114     |



```
In [17]: rf_pred = best_rf.predict(X_test)

print("\nRANDOM FOREST RESULTS")

print("Accuracy:",
      accuracy_score(y_test, rf_pred))

print("Precision:",
      precision_score(y_test, rf_pred))

print("Recall:",
      recall_score(y_test, rf_pred))

print("F1 Score:",
      f1_score(y_test, rf_pred))

print("\nClassification Report")
print(classification_report(y_test, rf_pred))

cm_rf = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm_rf,
            annot=True,
            fmt='d')
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

## RANDOM FOREST RESULTS

Accuracy: 0.9736842105263158

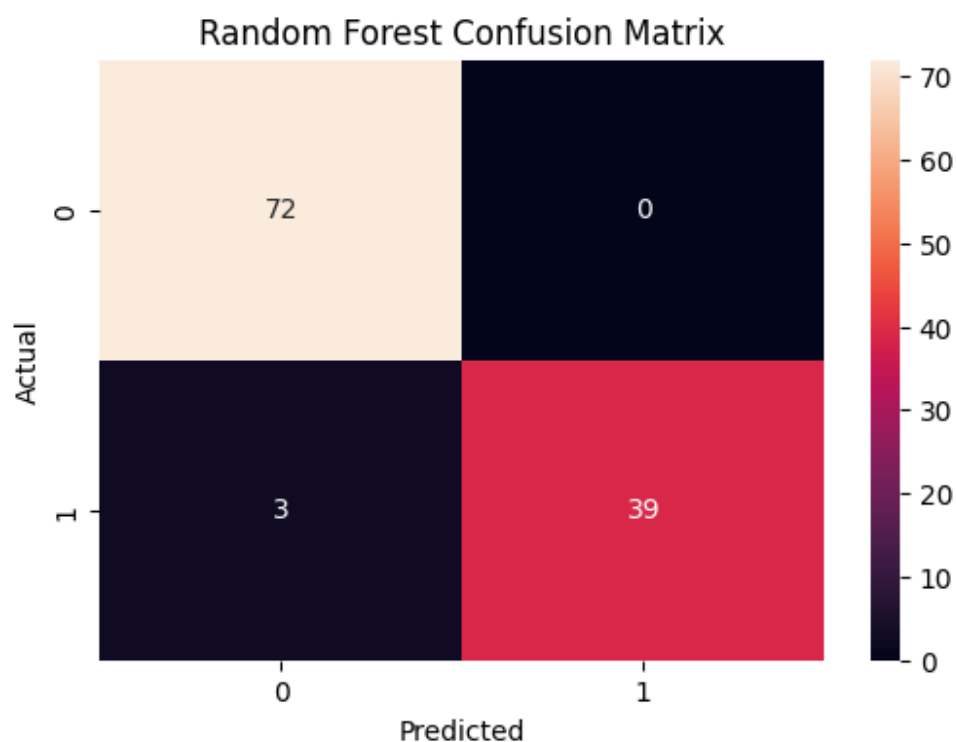
Precision: 1.0

Recall: 0.9285714285714286

F1 Score: 0.9629629629629629

## Classification Report

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.96      | 1.00   | 0.98     | 72      |
| 1            | 1.00      | 0.93   | 0.96     | 42      |
| accuracy     |           |        | 0.97     | 114     |
| macro avg    | 0.98      | 0.96   | 0.97     | 114     |
| weighted avg | 0.97      | 0.97   | 0.97     | 114     |



```
In [18]: dt_prob = best_dt.predict_proba(X_test)[: ,1]
rf_prob = best_rf.predict_proba(X_test)[: ,1]

fpr_dt, tpr_dt, _ = roc_curve(y_test,dt_prob)
fpr_rf, tpr_rf, _ = roc_curve(y_test,rf_prob)

auc_dt = auc(fpr_dt,tpr_dt)
auc_rf = auc(fpr_rf,tpr_rf)

plt.figure(figsize=(8,6))

plt.plot(
    fpr_dt,
    tpr_dt,
    label=f'Decision Tree AUC = {auc_dt:.4f}'
)
```

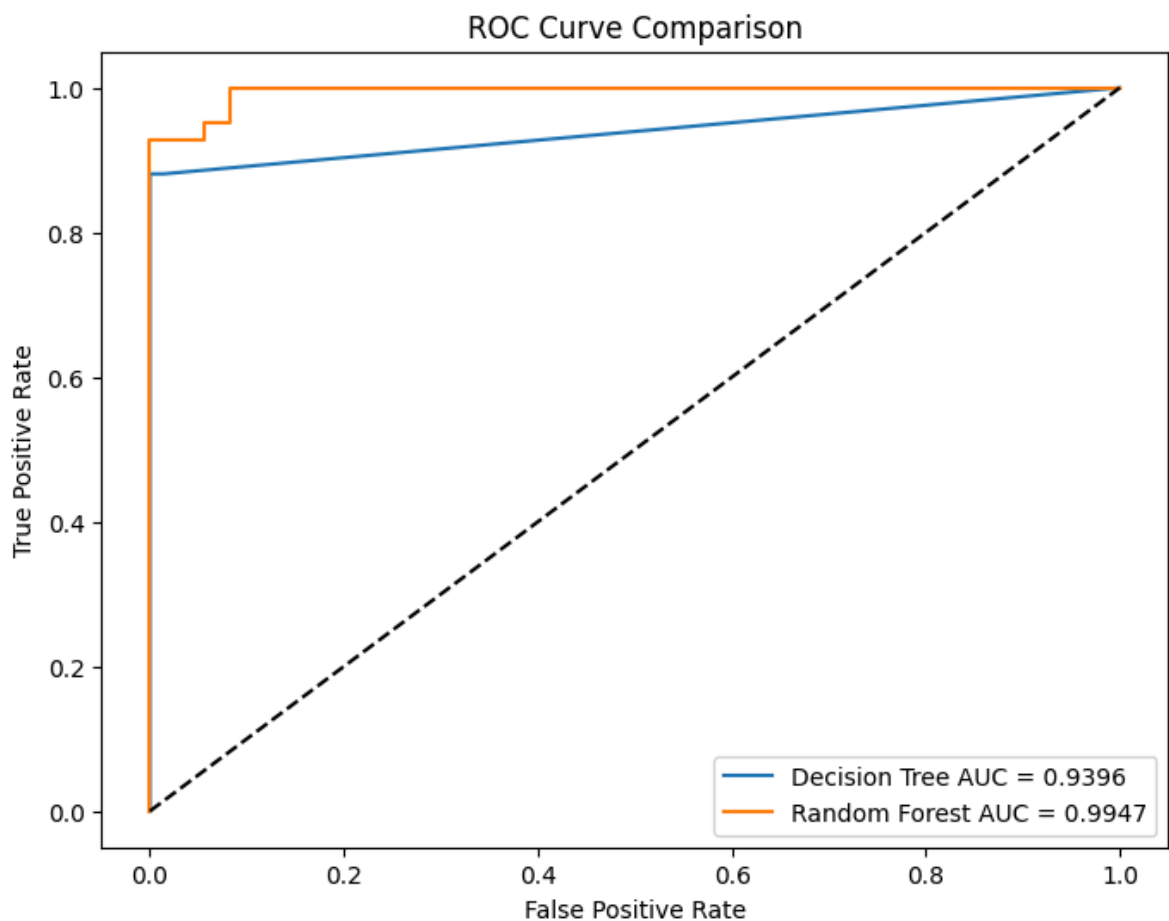
```
plt.plot(
    fpr_rf,
    tpr_rf,
    label=f'Random Forest AUC = {auc_rf:.4f}'
)

plt.plot([0,1],[0,1], 'k--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")

plt.legend()
plt.show()

print("\nDecision Tree AUC :", auc_dt)
print("Random Forest AUC :", auc_rf)
```



Decision Tree AUC : 0.9396494708994709  
 Random Forest AUC : 0.9947089947089948

## 6 Conclusion

Decision Tree and Random Forest classifiers were implemented and evaluated using the Wisconsin Diagnostic Breast Cancer dataset. Hyperparameter tuning was performed using GridSearchCV with five-fold cross-validation to identify suitable model configurations. The experimental results demonstrated that both models achieved high classification performance. However, the Random Forest classifier provided better generalization and stability due to its ensemble learning approach. The comparative analysis showed that Random Forest achieved superior cross-validation accuracy and ROC-AUC performance when compared to a single Decision Tree model.

## 7 Learning Outcomes

- Understood the working principles of Decision Tree classification.
- Learned how Random Forest combines multiple decision trees using ensemble learning.
- Gained experience in preprocessing and analyzing a real-world dataset.
- Performed hyperparameter tuning using GridSearchCV.
- Applied Stratified K-Fold cross-validation for robust model evaluation.
- Evaluated classification models using Accuracy, Precision, Recall, F1-Score, and ROC-AUC metrics.
- Analyzed the impact of hyperparameters on model performance and overfitting.
- Compared the effectiveness of single-tree and ensemble-tree classification models.